Code Quality Metrics

Metrics

- Cyclomatic complexity
- Test coverage
- Code duplication
- Code churn
- Technical debt
- Maintainability index
- Security
- Bug rate
- Standards compliance
- Documentation quality

Cyclomatic Complexity

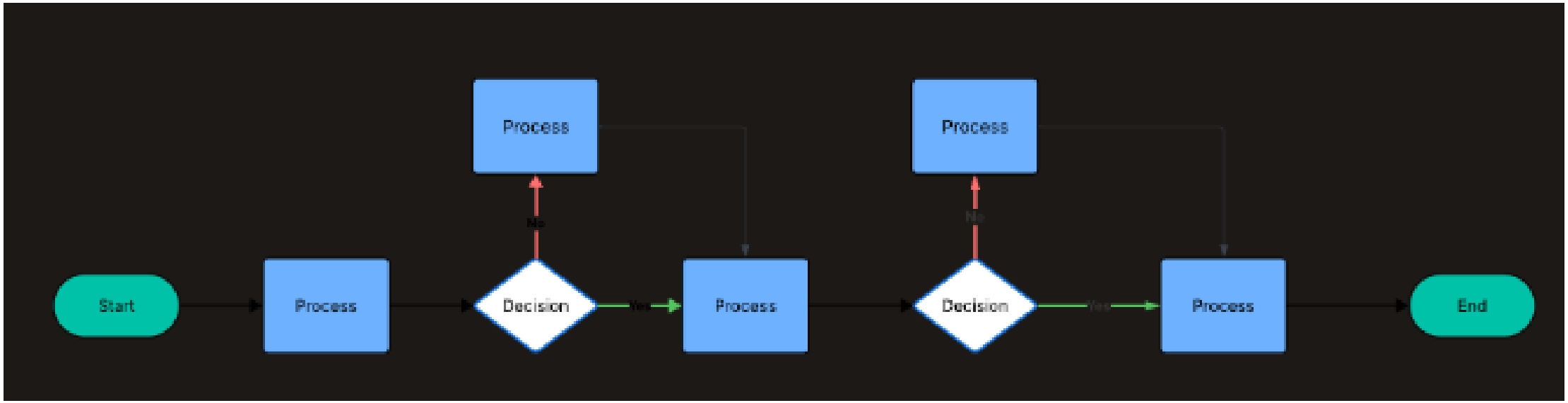
- The number of independent paths through a function
- Every branch increases complexity
  - Sometimes exponentially
- Harder to maintain
- As complexity increases, so do bugs

```
def my_function(self):
  if some_check():
    ... run this code ...
  else:
    ... run this code instead ...
```

Cyclomatic Complexity

- Calculating cyclomatic complexity usually involves a node-edge graph called a flow-graph

Cyclomatic Complexity



Cyclomatic Complexity

- Each arrow is an edge
- Each shape is a node
- M = E – N + 2
  - 1 - 10: simple procedure
  - 11 – 20: more complex, moderate risk
  - 21 – 50: Complex, high risk
  - > 59: untestable code, very high risk

Test Coverage

- Code not tested, by default should be considered broken
- Impossible to fully test our code

Test Coverage

- Equivalence partitioning: sampling of values that represent the valid collection
  - Boundary values
    - Extreme end of our valid range
  - Testing near boundaries is called **boundary value analysis**

Code Duplication

- Code duplication measures how much of your codebase is made up of repeated blocks of logic rather than shared, reusable pieces:
  - Duplication % = (Duplicated Lines / Total Lines) x 100
- Small amounts of duplication is unavoidable/expected
- High duplication usually is a sign that abstraction is missing:
  - Shared function
  - A base class
  - A utility module

Code Duplication

- DRY Principle – Don’t Repeat Yourself
- Pylint duplicate-code checker

Code Churn

- Measures how frequently a piece of code changes over time
- Churn = lines added + lines deleted + lines modified
- Fancy tools? Git?

Technical Debt

- Ward Cunningham
  - Describes the shortcuts we take to ship something quickly, with the understanding that we'll pay down the "principal" later by cleaning it up.
- Two axes: (Deliberate, Inadvertent), (Reckless, Prudent)

Technical Debt

	Deliberate	Inadvertent
Reckless	"We don't have time for tests."	"What's a design pattern?"
Prudent	"We'll ship now and refactor after launch."	"Now we know how we should have built it."

Technical Debt

- Measuring technical debt directly is difficult
- Technical debt ratio – SonarQube

$$\text{Technical Debt Ratio} = \frac{\text{Remediation Cost}}{\text{Development Cost}} \times 100$$

Technical Debt

- Under 5% - healthy
- 20% and above – “should we rewrite this?”

Maintainability Index

- Baking in cyclomatic complexity, code duplication, and lines of code together

$$MI = 171 - 5.2\ln(V) - 0.23(G) - 16.2\ln(LOC)$$

- V is Halstead Volume (how much information packed into the code, based on number of operators and operands)
- G is cyclomatic Complexity
- LOC is number lines of code

Maintainability Complex

- 85-100: highly maintainable
- 65-84: Moderately maintainable
- Below 65: difficult to maintain

Security

- How vulnerable our code base is to attack
- Static Application Security Testing tools can be used.
  - SQL queries
  - Hard-code credentials
  - Use of weak or deprecated cryptographic functions
  - Unsanitized user input flowing into a shell command
- Bandit

Security

- Number of known vulnerabilities: often cross-referenced against the CVE (Common Vulnerabilities and Exposure) database for third-party dependencies
- OWASP Top 10 coverage: whether thee codebase has been checked against the most common categories of web application vulnerability
- Mean time to patch: how quickly a known vulnerability is fixed once discovered

Bug Rate

- Bug rate (aka defect density) measures how many bugs show up in a piece of code, normalixed by its sizw so that a 10,000 line file isn’t unfairly compared to 100 line code

$$\text{Defect Density} = \frac{\text{Number of Defects}}{\text{Thousand Lines of Code (KLOC)}}$$

Standards Compliance

- PEP8
- pylint

Documentation Quality

- Docstring Coverage
- Comment Density
- Freshness